Appendix J: Implementation of CSS variance smoothing Fay-Herriot model

In this vignette we define the Complex Sampling Single-χ^2 (CSS) variance smoothing Fay-Herriot model and use it to estimate height-for-age z-score Admin-2 means from the 2022 Kenya DHS. Note that this is model is specifically for continuous outcomes.

Let $\hat{\theta}_i$ and $\hat{V}_i$ denote the design-based mean and variance estimate for Admin-2 area $i = 1, ..., K$. Then the Fay-Herriot model is defined by

$$\hat{\theta}_i \mid \theta_i, \sigma_i^2 \sim \mathcal{N}\left(\theta_i, \frac{\sigma_i^2}{n_i^+}\right), \qquad \hat{V}_i \mid \sigma_i^2 \sim \frac{\sigma_i^2}{n_i^+ \cdot d_i}\chi^2_{d_i}$$

$$\theta_i = \boldsymbol{X}_i^T\boldsymbol{\beta} + b_i, \qquad \log(\sigma_i^2) = \boldsymbol{Z}_i^T\boldsymbol{\eta} + g_i$$

where

- n_i^+ is the number of sampled individuals in area i
- $d_i =$ (number of sampled clusters in area i) $-$ (number of strata area i is contained in)
- $\boldsymbol{X}_i$ and $\boldsymbol{Z}_i$ are covariate vectors
- $\boldsymbol{b} = (b_1, ..., b_K)$ and $\boldsymbol{e} = (e_1, ..., e_K)$ are area-level random effects, which we will choose to model as BYM2 effects

Setup

First, we load the survey data, covariates, and spatial information.

```
library(sf)
library(spdep)
library(haven)
library(tidyverse)
library(INLA)
library(cmdstanr)
library(ggpubr)
library(ggplot2)
library(scico)

# load survey data
load(file=paste0('KEN_2022_haz_data.rda'))

# load covariates
load(file='KEN_admin2_covariates.rda')

# load geometry
poly.adm2 <- st_read(dsn = paste0('gadm41_KEN_shp'),
                     layer = paste0("gadm41_KEN_2"), options = "ENCODING=UTF-8")
```

```
poly.adm2$admin2 <- 1:nrow(poly.adm2)
poly.adm2$admin2.char <- paste0('admin2_',1:nrow(poly.adm2))
n_admin2 <- nrow(poly.adm2)

#adjacency matrix
admin2.mat <- nb2mat(poly2nb(poly.adm2), zero.policy = TRUE)
colnames(admin2.mat) <- rownames(admin2.mat) <- poly.adm2$admin2.char

#nodes for ICAR
admin2.nbs <- poly2nb(poly.adm2)
nodes2 <- NULL
for(i in 1:n_admin2){
  nodes2 <- rbind(nodes2,data.frame(node1=i,node2=as.numeric(admin2.nbs[[i]])))
}
nodes2 <- nodes2[nodes2$node1<nodes2$node2,]

# get scale for ICAR
Q.admin2 <- -admin2.mat
Q.admin2 <- sapply(1:nrow(Q.admin2),function(i){sum(I(Q.admin2[i,]!=0))})*Q.admin2
diag(Q.admin2) <- sapply(1:nrow(Q.admin2),function(i){sum(I(Q.admin2[i,]!=0))})
diag(Q.admin2)[diag(Q.admin2)==0] <- 1
Q2_inv <- INLA:::inla.ginv(as.matrix(Q.admin2))
Q2_scaled <- INLA::inla.scale.model(Q.admin2,
                                    constr=list(A=matrix(1,nrow=1,ncol=n_admin2), e=0))
```

Get design-based estimates

Next, we get design-based mean and variance estimates, $(\hat{\theta}_i, \hat{V}_i)$, for each area of interest.

```
options(survey.lonely.psu = "adjust")
my.svydesign <- survey::svydesign(ids = stats::formula("~cluster"),
                                  strata = ~v023, nest = T, weights = ~wt, data = haz_dat)

# define function to get design-based estimates
admin2.HT.withNA <- function(which.area) {
  admin2 <- NULL
  tmp <- subset(my.svydesign, (admin2 == as.character(which.area)))

  if (dim(tmp)[1] == 0) {
    return(c(which.area,rep(NA, 2)))
  } else {
    lm.ob <- survey::svymean(haz ~ 1, design = tmp)
    return(c(which.area, lm.ob[1], vcov(lm.ob)))
  }
}
# apply function  to each area
x <- mapply(admin2.HT.withNA, which.area = 1:n_admin2)
admin2.dir <- data.frame(t(x))
colnames(admin2.dir) <- c('admin2','mean','variance')

# add covariates
admin2.dir <- merge(admin2.dir,cmat_admin2)
admin2.dir <- admin2.dir[order(admin2.dir$admin2),]
```

```
# remove design-based estimates with near-zero variance
admin2.dir.stable <- admin2.dir
admin2.dir.stable$variance <- ifelse(admin2.dir.stable$variance < 1e-5,NA,admin2.dir.stable$variance)
admin2.dir.stable$mean <- ifelse(is.na(admin2.dir.stable$variance),NA,admin2.dir.stable$mean)
```

Fit CSS variance smoothing Fay-Herriot model

Now we fit the CSS variance smoothing Fay-Herriot model. We define a list of arguments and then compile and fit the model in Stan. Note that in this example we only use covariates in the mean model, not the variance model. For both the latent mean and variance models we can use either BYM2 or IID random effects: we choose to use BYM2 for both here.

```
# specify parameters for the CSS smoothing model
data_areas <- which(!is.na(admin2.dir.stable$mean))
CSS_params <- haz_dat %>% filter(admin2 %in% data_areas) %>%
  group_by(admin2) %>%
  reframe(Cons = 1/n(),m = length(unique(cluster)), H = length(unique(v025))) %>%
  mutate(df=pmax(1,m-H))

mean_covariates <- c('urb_frac','density_log', 'nt_lights_log', 'tthc_log', 'precip', 'temp', 'elev')
mean_model_matrix <- cbind(rep(1,nrow(cmat_admin2)),cmat_admin2[,mean_covariates])

data_list = list(m=n_admin2, # number of areas
                 m_data=length(data_areas), # number of areas with data
                 data_areas = data_areas, # indices of areas with data
                 y=admin2.dir.stable$mean[data_areas], # direct mean estimates
                 v_hat_scaled = admin2.dir.stable$variance[data_areas]*CSS_params$df, # direct variance
                 df=CSS_params$df, # degrees of freedom for variance
                 Cons = CSS_params$Cons, # constant, connecting sigma_i^2 and V_i
                 X = mean_model_matrix,# covariate matrix for mean
                 p_mean = ncol(mean_model_matrix), # number of mean covariates
                 Z = cbind(rep(1,n_admin2)), # covariate matrix variance
                 p_var = 1, # number of variance covariates
                 bym2_mean = 1, # should random effects on mean be BYM2 or IID?
                 bym2_var = 1, # should random effects on variance be BYM2 or IID?
                 N_edges = nrow(nodes2), # params for ICAR model
                 node_1 = nodes2$node1, # params for ICAR model
                 node_2 = nodes2$node2, # params for ICAR model
                 car_scale = Q2_scaled[1,1]/Q.admin2[1,1]) # params for ICAR model

mod_CSS <- cmdstan_model(stan_file = "../Stan/CSS.stan")

CSS.fit <- mod_CSS$sample(
  data = data_list,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000,
  adapt_delta = 0.99,
  show_messages = F, #turn on for troubleshooting
  show_exceptions = F, #turn on for troubleshooting
  refresh=00)
```

```r
# here we just report the area means, but you can select any model parameters to summarize
theta_fit <- as.matrix(unclass(CSS.fit$draws(variables = c("theta"), format = "matrix")))
CSS_res <- data.frame(area = 1:n_admin2,
                      mean = apply(theta_fit,2,mean),
                      sd = apply(theta_fit,2,sd),
                      upper = apply(theta_fit,2,quantile,probs=0.95),
                      lower = apply(theta_fit,2,quantile,probs=0.05))
```

Fit standard Fay-Herriot model

Now we fit an analgous standard Fay-Herriot model, for comparison. The mean model is the same as in CSS variance smoothing model.

```r
hyperpc.bym2 = list(prec = list(prior = "pc.prec", param = c(1, 0.01)),
                    phi = list(prior = "pc", param = c(0.5, 0.5)))

formula_str <- paste0("mean ~ ", paste(mean_covariates, collapse = " + "),
                      " + f(admin2, model='bym2', graph=admin2.mat, hyper=hyperpc.bym2, scale.model=T, a

std.fit <- inla(as.formula(formula_str),
  family = "gaussian",
  data = admin2.dir.stable,
  scale = 1 / variance,
  control.fixed = list(quantiles=c(0.05,0.5,0.95)),
  control.family = list(hyper = list(prec = list(initial = log(1), fixed = TRUE))),
  control.compute = list(config = TRUE)
)

cs <- std.fit$misc$configs$contents$tag
cs <- cs[cs != "Predictor"]
select <- list()
for (i in 1:length(cs)) {
    select[[i]] <- 0
    names(select)[i] <- cs[i]
  }
samp <- INLA::inla.posterior.sample(n = 1000, result = std.fit,
                                    intern = TRUE, selection = select)
std.samples  <- matrix(NA,1000,length(samp[[1]]$latent))
for(i in 1:1000){
    std.samples [i,] <- samp[[i]]$latent
}
colnames(std.samples) <- row.names(samp[[1]]$latent)

X <- model.matrix(as.formula(paste0("~ ", paste(mean_covariates, collapse = " + "))),
                  data = admin2.dir.stable)
n_fixed <- ncol(X)
intercept_col <- 2*n_admin2 + 1
fixed_cols <- (intercept_col):(intercept_col + (n_fixed - 1))
eta.samples <- std.samples[, fixed_cols]%*% t(X[, , drop = FALSE]) + std.samples[, 1:n_admin2]

std.res <- data.frame(mean  = apply(eta.samples,2,mean),
           sd = apply(eta.samples,2,sd),
           lower = apply(eta.samples,2,quantile,prob=0.05),
```

```
            upper = apply(eta.samples,2,quantile,prob=0.95),
            area = 1:n_admin2,
            model='std')
```

Compare Fay-Herriot models

Now we can compare mean and uncertainty estimates from the two Fay-Herriot models.

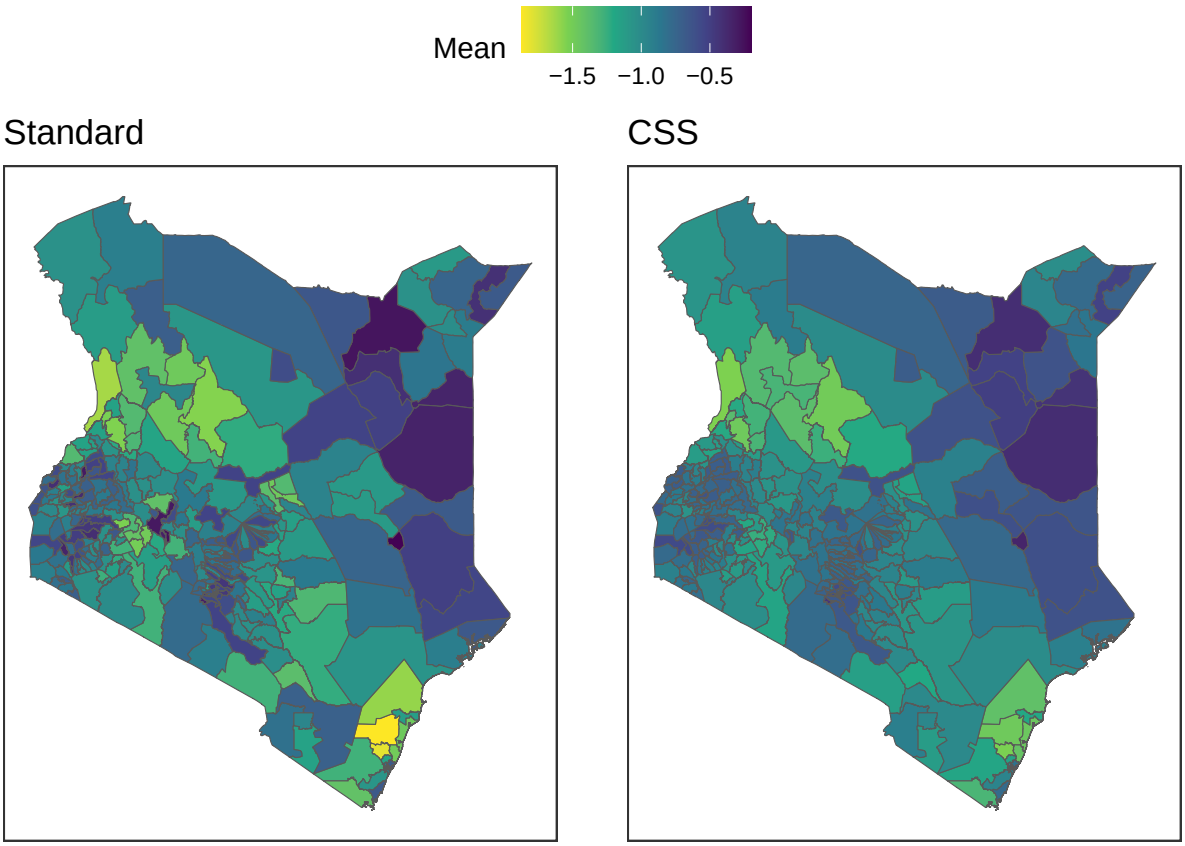
```
clean_map_theme <- ggplot() + theme_bw() +
  theme(axis.text.x=element_blank(),
        axis.ticks.x=element_blank(),
        axis.text.y=element_blank(),
        axis.ticks.y=element_blank(),
        panel.grid.major = element_blank(),
        panel.grid.minor = element_blank())

lims <- range(std.res$mean,CSS_res$mean)

poly.adm2$std.mean <- std.res$mean
poly.adm2$CSS.mean <- CSS_res$mean

m1 <- clean_map_theme + geom_sf(data = poly.adm2,aes(fill=std.mean)) +
  ggtitle('Standard') +
  scale_fill_viridis_c(name = 'Mean',direction = -1,limits = lims)
m2 <- clean_map_theme + geom_sf(data = poly.adm2,aes(fill=CSS.mean)) +
  ggtitle('CSS') +
  scale_fill_viridis_c(name = 'Mean',direction = -1,limits = lims)

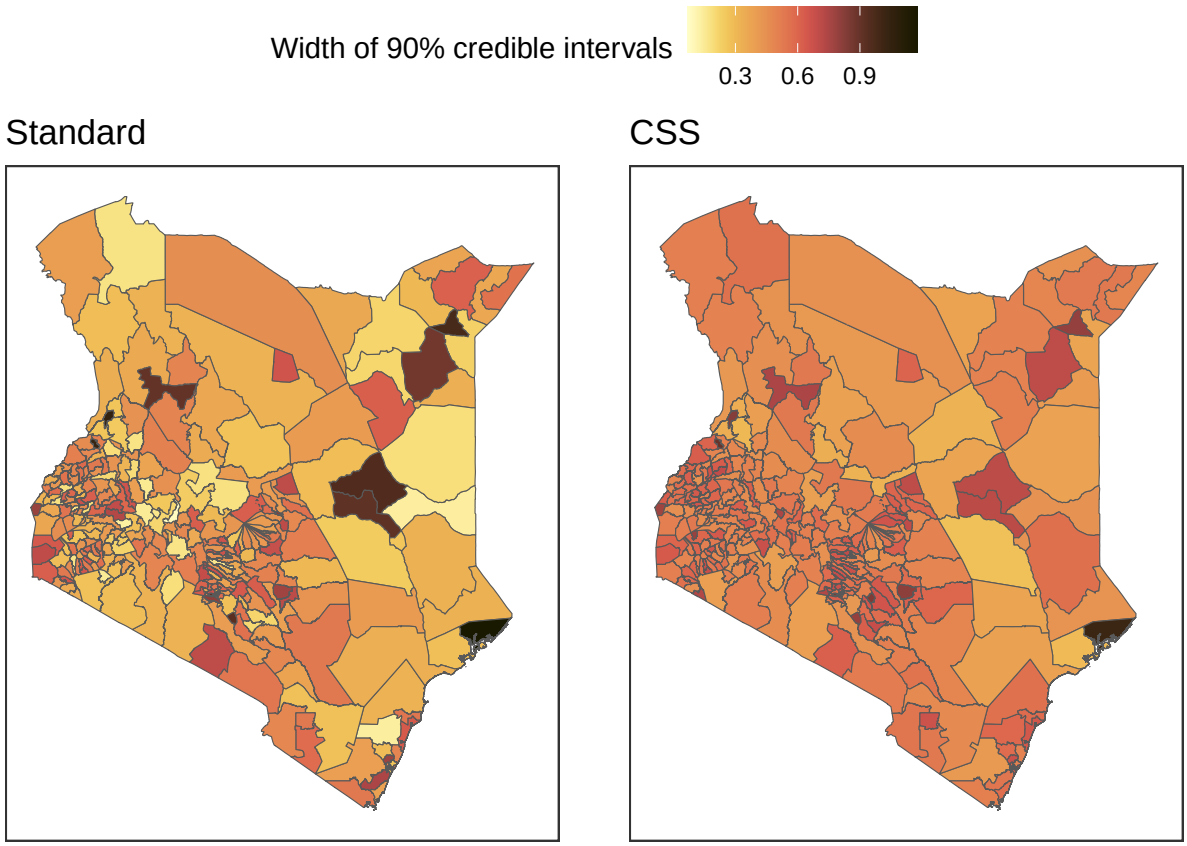
ggarrange(plotlist = list(m1,m2),common.legend = T,nrow=1)
```




```
poly.adm2$std.ciwidth <- std.res$upper - std.res$lower
poly.adm2$CSS.ciwidth <- CSS_res$upper - CSS_res$lower
lims <- range(poly.adm2$std.ciwidth,poly.adm2$CSS.ciwidth)

v1 <- clean_map_theme + geom_sf(data = poly.adm2,aes(fill=std.ciwidth)) +
  ggtitle('Standard') +
  scale_fill_scico(name = 'Width of 90% credible intervals',
                   palette = 'lajolla',direction = -1,limits = lims)
v2 <- clean_map_theme + geom_sf(data = poly.adm2,aes(fill=CSS.ciwidth)) +
  ggtitle('CSS') +
  scale_fill_scico(name = 'Width of 90% credible intervals',
                   palette = 'lajolla',direction = -1,limits = lims)

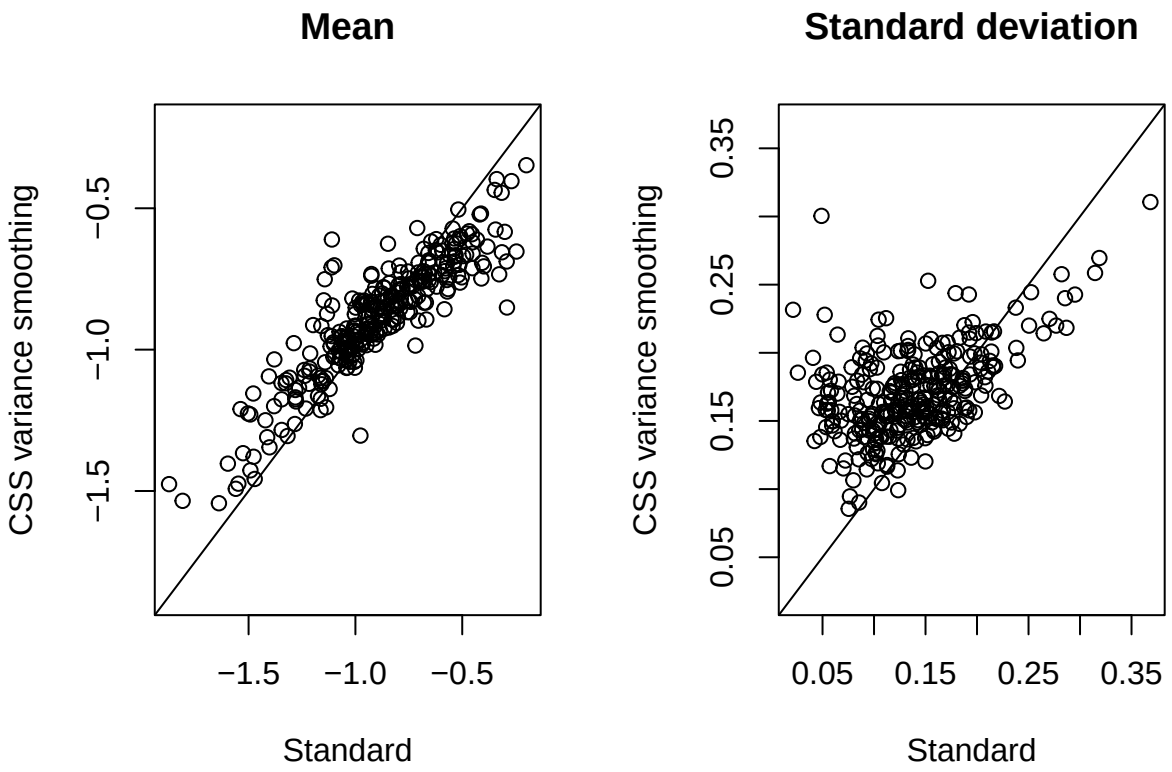
ggarrange(plotlist = list(v1,v2),common.legend = T,nrow=1)
```




```
par(mfrow=c(1,2))

lims <- range(std.res$mean,CSS_res$mean)
plot(std.res$mean,CSS_res$mean,
     main='Mean',xlab = 'Standard',ylab = 'CSS variance smoothing',
     xlim=lims,ylim=lims)
abline(0,1)

lims <- range(std.res$sd,CSS_res$sd)
plot(std.res$sd,CSS_res$sd,
     main='Standard deviation',xlab = 'Standard',ylab = 'CSS variance smoothing',
     xlim=lims,ylim=lims)
abline(0,1)
```



Troubleshooting

- check that the data does not include any NAs
- for the Stan parameters, make sure the first columns of X and Z are 1s
- make sure all Stan parameters don't contain NAs and are of correct dimension, using the vignette as a guide